

App Deck – Technische Dokumentation

Überblick

App Deck ist eine in Java Swing entwickelte Desktop-Anwendung für macOS, die als frei konfigurierbare Schaltflächen-Leiste (a la Stream Deck) fungiert. Sie erlaubt das Starten von URLs, Programmen, Ordnern und das Kopieren von Text in die Zwischenablage über ein Raster von 6x4 Schaltflächen pro Seite mit beliebig vielen Seiten und optionalen Unter-Ordnern.

Systemvoraussetzungen

- Java 21+
- macOS (für .app-Icons via sips, App-Erkennung via osascript und .app-Bundle-Erstellung via jpackage)

Architektur

Paketstruktur

src/main/java/streamdeck/

StreamDeckApp.java – Hauptklasse (GUI, Logik, Icons, Drag & Drop)
ButtonConfig.java – Datenmodell für eine Schaltfläche
ConfigLoader.java – JSON-Persistenz (Gson)

src/main/resources/streamdeck/

app-icon.png – Eingebettetes App-Icon

Datenmodell (ButtonConfig)

Jede Schaltfläche wird durch ein ButtonConfig-Objekt repräsentiert:

```

public class ButtonConfig {
    private String label;           // Anzeigetext
    private String type;           // URL | PROGRAM |
                                   FOLDER | COPY
    private String
        target;                   // URL, Pfad, Kommando oder Text
    private List<List<ButtonConfig>> pages; // nur bei FOLDER:
                                   Unterseiten
}

```

Konfigurationsformat (ConfigLoader)

Die Konfiguration wird als JSON gespeichert, ein Array von Seiten (Arrays von ButtonConfig-Objekten):

```

[
  [
    { "label": "GitHub", "type": "URL", "target": "https://
      github.com" },
    { "label": "Terminal", "type": "PROGRAM", "target": "open -a
      Terminal" },
    { "label": "Dokumente", "type": "URL", "target": "file:///
      Users/name/Dokumente" },
    { "label": "Passwort", "type": "COPY", "target":
      "meinPasswort123" }
  ],
  [
    { "label": "Projekt", "type": "FOLDER", "target": "",
      "pages": [
        [
          { "label": "Build", "type": "URL", "target": "https://
            jenkins.example.com" },
          { "label": "Git", "type": "URL", "target": "https://
            github.com/mein/projekt" }
        ]
      ]
    }
  ]
]

```

Der ConfigLoader unterstützt abwärts kompatible flache JSON-Arrays (einzelnes Array für Seite 0), die automatisch in das Mehrseiten-Format überführt werden.

GUI-Komponenten

Raster

- 6 Spalten x 4 Zeilen = 24 Plätze pro Seite
- Konfigurierbar über Konstanten COLS und ROWS
- Quadratische Schaltflächen mit 120x120 Pixel
- 14px abgerundete Ecken mit Schlagschatten (Klasse RoundedShadowBorder)
- Hellgrauer Gradient (248,248,250 nach 225,225,230) mit blaulichem Hover-Effekt (230,245,255 nach 200,225,240)
- Gedrückter Zustand: dunklerer Gradient (210,210,215 nach 185,185,190)
- Leere Schaltflächen: dezent dunkel (55,55,60), nahezu unsichtbar auf dem Hintergrund (50,50,55)
- 10px Abstand zwischen den Schaltflächen, 10px Außenabstand
- Text: fett, 10pt, zentriert unter dem Icon

Navigationsschaltflächen

Position	Beschreibung
Unten rechts (immer)	Vorwärts ► (28pt)
Unten links (ab Seite 1)	Rückwärts ◀ (28pt)
Unten links (Ordner, Seite 0)	Rückkehr ← (48pt)
Vorletzte Position (Ordner, Seite >0)	Rückkehr ← (48pt) neben ◀

Versionslabel

Ganz unten: "V1.0 vom 16.05.26" in 9pt, grau (150,150,150).

Funktionen im Detail

Button-Typen

URL: Öffnet die angegebene URL im Standardbrowser (`Desktop.getDesktop().browse()`). Unterstützt `file://`-Pfade für lokale Dateien und Verzeichnisse.

PROGRAM: Startet ein Programm mit `Runtime.getRuntime().exec()`. Akzeptiert: - `open -a AppName` – macOS-App - `open "/Applications/App.app"` – Pfad mit Leerzeichen - `/Pfad/zur/App.app` – Direkter Pfad - Beliebige Shell-Kommandos

FOLDER: Erzeugt eine Unterseiten-Struktur. Beim Klick wird in den Ordner navigiert, der eigene Seiten mit bis zu 24 Schaltflächen pro Seite haben kann. Die Rückkehr erfolgt über die `←`-Schaltfläche. Ordnertiefe ist auf eine Ebene beschränkt (verschachtelte Ordner werden ignoriert).

COPY: Kopiert den Zielfeld-Text in die System-Zwischenablage und zeigt kurz "Kopiert!" auf der Schaltfläche an.

Rechtsklick-Menu

- **Bearbeiten** – Dialog zum Ändern von Label, Typ und Ziel (URL/PROGRAM/FOLDER/COPY)
 - Auto-Erkennung des Typs während der Eingabe
 - Label-Vorschlag aus URL-Domain, Dateipfad oder Programmname
 - Dateiauswahldialog mit Tastatur-Navigation (Buchstaben springen zum passenden Eintrag)
 - FOLDER: Zielfeld deaktiviert
 - COPY: Mehrzeiliges Textfeld (5 Zeilen, Zeilenumbruch)
- **Ordner anlegen** – Erzeugt einen neuen FOLDER-Button
- **Entfernen** – Löscht die Schaltfläche (null im Seiten-Array)

Laufende Apps erkennen

Alle 5 Sekunden wird die Liste der laufenden Prozesse via `osascript` und `ps -ef` abgefragt. Programme mit einem laufenden Prozess erhalten einen grünen Rahmen (`ROUNDED_BORDER_RUNNING`, 4px, RGB 0,180,0). Die Erkennung gleicht sowohl den Prozessnamen als auch die Kommandozeile ab, um auch Apps mit abweichenden Namen (z.B. VS Code als 'Code') zu erfassen.

Long-Press zum Beenden

Ein PROGRAM-Button muss >800ms gedrückt gehalten werden, um die App via `osascript` zu beenden. Der grüne Rahmen wird sofort entfernt, und die App erscheint für 12s nicht mehr als laufend (Kühlung, bis der OS-Beendigungsprozess abgeschlossen ist).

Drag & Drop

- Drag-Schwelle: 5px (reagiert schon bei kleinen Bewegungen)
- Ghost-Fenster (JWindow) zeigt das Schaltflächen-Bild während des Ziehens
- Hand-Cursor während des Ziehens
- Ablegen auf eine andere Schaltfläche: Tausch der Positionen
- Ablegen auf eine FOLDER-Schaltfläche: Verschieben in den ersten leeren Slot des Ordners
- Ablegen auf die Rückkehr-Schaltfläche (während Ordner geöffnet): Verschieben zurück auf die Root-Seite (erster freier Platz)
- Long-Press-Timer wird bei Drag-Start abgebrochen
- Leichte Persistenz: nur die beiden betroffenen Schaltflächen werden aktualisiert, volles `saveAndRefresh()` nur bei Bearbeitungen

Icon-Ladung

Icons werden asynchron (Hintergrund-Thread) geladen, um die EDT nicht zu blockieren:

1. **Cache** (ConcurrentHashMap) – type + "::" + target als Key
2. **PROGRAM**: ShellFolder (Reflection) → FileSystemView → sips für .icns (temp PNG mit Pfad-Hash) → lokale favicon.svg → Globe-Fallback
3. **URL**: Google Favicon-Service (<https://www.google.com/s2/favicons>) → lokale favicon.svg → Globe-Fallback
4. **Globe-Fallback** (48x48): Blauer Kreis mit Breiten-/Langengrad-Linien (Java2D)
5. **FOLDER** (48x48): Gelb-oranger Ordner mit Lasche (Java2D)
6. **COPY** (48x48): Blaue Zwischenablage mit Textzeilen (Java2D)
7. Stale-Callback-Guard: Vor dem Setzen des Icons wird geprüft, ob die Schaltfläche noch die gleiche Konfiguration hat

Programmatische Icons (Java2D)

Drei Icons werden ohne externe Dateien zur Laufzeit generiert:

- **Globe**: Blauer Kreis (70,140,220) mit weißen Hilfslinien für Äquator und Nullmeridian
- **Ordner**: Gelb-oranger Gradient (245,215,85 bis 210,175,50) mit braunem Tab und Umriss
- **Kopie**: Hellblaue Briefklammer oben, dunkelblaues Rechteck (100,120,200) mit weißen Textzeilen

macOS .app-Bundle

Das Skript `build-app.sh` erzeugt ein .app-Bundle via `jpackage`:

```
./build-app.sh  
open "App Deck.app"
```

Eigenschaften: - Bundle-ID: `com.dobronski.appdeck` - Custom .icns-Icon
(5x3 farbiges Raster) - Dock-Name via
`System.setProperty("apple.awt.application.name", "App Deck")` - Dock-
Icon via `Taskbar.setTaskbarIcon()`

Build und Ausführung

JAR (einfach)

```
mvn package -q  
java -jar target/streamdeck-1.0-SNAPSHOT.jar [config.json]
```

macOS .app Bundle

```
./build-app.sh  
open "App Deck.app"
```

Konfigurationspfad

1. Kommandozeilen-Argument, falls angegeben
2. `config.json` im aktuellen Verzeichnis
3. `~/Library/Application Support/App Deck/config.json` (wird automatisch angelegt)

Abhängigkeiten

- **Gson 2.11.0** – JSON-Serialisierung (`com.google.code.gson:gson`)
- Keine weiteren externen Bibliotheken (reines Java Swing + macOS-native Tools)

Wichtige Konstanten

Konstante	Wert	Beschreibung
COLS	6	Spalten im Raster
ROWS	4	Zeilen im Raster
BUTTON_SIZE	120	Seitenlänge der quadratischen Buttons (px)
BOTTOM_ROW_START	$(ROWS-1)*COLS$	Index der ersten Schaltfläche in der unteren Zeile
ICON_SIZE	48	Zielgröße der Icons (px)
DRAG_THRESHOLD	5	Pixel für Drag-Erkennung
ARC	14	Rundungsradius der Buttons
SHADOW	3	Schlagschatten-Ver-satz

Index-Berechnungen

Die Navigationselemente belegen Slots in der unteren Zeile:

- Letzter Slot ($COLS*ROWS-1$): immer Vorwärts ►
- Erster Slot der unteren Zeile ($BOTTOM_ROW_START$): Rückwärts ◀ (wenn Seite > 0) oder Rückkehr ← (Ordner Seite 0)
- Vorletzter Slot ($COLS*ROWS-2$): Rückkehr ← (Ordner Seite > 0)

`gridToPageIndex()` und `pageToGridIndex()` bilden zwischen Gitter-Position und Seiten-Index ab, wobei Navigations-Slots übersprungen werden.

Drag & Drop in Ordner

Beim Ablegen einer Schaltfläche auf einen FOLDER-Button: 1. Die Quell-Konfiguration wird in den ersten leeren Slot der ersten Seite des Ordners eingefügt 2. Der Quell-Slot wird auf null gesetzt 3. Nur die beiden betroffenen Schaltflächen werden neu gezeichnet

Beim Ablegen auf die Rückkehr-Schaltfläche (aus einem Ordner heraus): 1. Die Quell-Konfiguration wird in den ersten leeren Slot der Root-Seite eingefügt 2. Der Ordner wird automatisch verlassen

Running-App-Erkennung

```
isAppRunning(appKey) = runningApps.contains(appKey)
                        || runningCmdLines.contains(appKey)
```

- runningApps: via osascript ermittelte Prozessnamen
- runningCmdLines: via ps -ef ermittelte Kommandozeilen
- killedApps: via Long-Press beendete Apps (12s Kühlung)

Long-Press-Timing

Phase	Dauer	Aktion
Gedrückt halten	>800ms	osascript quit, Rahmen entfernen, App in killedApps
Kühlung	12s	App erscheint nicht als laufend, damit der Beendigungsprozess abgeschlossen werden kann

Beispielkonfiguration

Eine vollständige Konfiguration mit zwei Seiten und einem Ordner:

```
[
  [
    { "label": "GitHub", "type": "URL", "target": "https://github.com" },
    { "label": "Terminal", "type": "PROGRAM", "target": "open -a Terminal" },
    { "label": "Projekte", "type": "FOLDER", "target": "",
      "pages": [
        [
          { "label": "Build", "type": "URL", "target": "https://jenkins.example.com" },
          { "label": "Wiki", "type": "URL", "target": "https://wiki.example.com" }
        ]
      ]
    }
  ]
]
```



```

    ]
  ]}
],
[
  { "label": "Notizen", "type": "PROGRAM", "target": "open -a
    Notes" },
  { "label": "Codeschnipsel", "type": "COPY", "target":
    "System.out.println(\"Hello World\");" }
]
]

```

Icon-Generierung

Das App-Icon (5x3 farbiges Raster auf dunklem Hintergrund) wird durch `icons/make-icon.sh` generiert, das ein temporäres Java-Programm kompiliert und ausführt, um ein 1024x1024 PNG zu erzeugen, das dann via `sips` und `iconutil` in ein `.icns`-Format gebracht wird.

```
./icons/make-icon.sh
```

Erzeugt: - `icons/app-icon.png` (1024x1024) - `icons/app-icon.icns` (macOS Icon-Format)